EGRefine: An Execution-Grounded Optimization Framework for Text-to-SQL Schema Refinement

Jiaqian Wang
Xidian University
Xi'an, Shaanxi, China
24151111272@stu.xidian.edu.cn

Yutao Qi*
Xidian University
Xi'an, Shaanxi, China
ytqi@xidian.edu.cn

Wenjin Hou
Xidian University
Xi'an, Shaanxi, China
houwenjin412@gmail.com

Yu Pang
Xidian University
Xi'an, Shaanxi, China
24151213784@stu.xidian.edu.cn

Rui Yang
Xidian University
Xi'an, Shaanxi, China
ryang_2@stu.xidian.edu.cn

ABSTRACT

Text-to-SQL enables non-expert users to query databases in natural language, yet real-world schemas often suffer from ambiguous, abbreviated, or inconsistent naming conventions that degrade model accuracy. Existing approaches treat schemas as fixed and address errors downstream. In this paper, we frame schema refinement as a constrained optimization problem: find a renaming function that maximizes downstream Text-to-SQL execution accuracy while preserving query equivalence through database views. We analyze the computational hardness of this problem and, since exact optimization over its exponential search space is intractable, instantiate a column-wise greedy decomposition as EGRefine: a four-phase pipeline that screens ambiguous columns, generates context-aware candidate names, verifies them through execution-grounded feedback, and materializes the result as non-destructive SQL views. The pipeline carries two structural properties: column-local non-degradation, ensured by the conservative selection rule in the verification phase, and database-level query equivalence, ensured by the view-based materialization phase. Together they make the resulting refinement empirically regression-resistant, with a per-column local guarantee; cross-column and prompt-level interactions are handled experimentally rather than analytically. Across controlled schema-degradation, real-world, and enterprise benchmarks, EGRefine recovers accuracy lost to schema naming noise when naming is an active bottleneck. Its applicability diagnostics identify settings in which other bottlenecks dominate, while refined schemas transfer across model families to enable refine-once, serve-many-models deployment. Code and data are publicly available at https://github.com/ai-jiaqian/EGRefine.

KEYWORDS

Text-to-SQL, Schema Refinement, Execution Feedback, Database Views, Optimization

PVLDB Reference Format:
Jiaqian Wang, Yutao Qi, Wenjin Hou, Yu Pang, and Rui Yang. EGRefine: An Execution-Grounded Optimization Framework for Text-to-SQL Schema Refinement. PVLDB, XX(X): XXX-XXX, 20XX.
doi:XX.XX/XXX.XX

*Corresponding author.
This work is licensed under the Creative Commons BY-NC-ND 4.0 International License. Visit https://creativecommons.org/licenses/by-nc-nd/4.0/ to view a copy of this license. For any use beyond those covered by this license, obtain permission by emailing info@vldb.org. Copyright is held by the owner/author(s). Publication rights licensed to the VLDB Endowment.
Proceedings of the VLDB Endowment, Vol. XX, No. X ISSN 2150-8097.
doi:XX.XX/XXX.XX

PVLDB Artifact Availability:
The source code, data, and/or other artifacts have been made available at https://github.com/ai-jiaqian/EGRefine.

1 INTRODUCTION

Text-to-SQL—the task of translating natural-language questions into executable SQL queries—has emerged as a pivotal technology for democratizing data access [22, 28]. Driven by rapid advances in large language models (LLMs), recent systems have achieved impressive performance on standard benchmarks [18, 33, 41, 47], with execution accuracies exceeding 85% on the widely used Spider dataset [52].

However, a persistent gap remains between benchmark performance and production reliability. Real-world databases are frequently characterized by inconsistent naming standards, ad-hoc abbreviations, and sparse documentation [15, 37]. Columns named `A2`, `nm`, `sal`, or `dt` carry little semantic information, forcing Text-to-SQL models to rely on fragile heuristic cues during schema linking [48]. Diagnostic evaluations confirm that even state-of-the-art systems suffer substantial accuracy degradation when schema names are perturbed or obfuscated [4, 26], establishing schema-level naming quality as a critical yet underexplored bottleneck for robust Text-to-SQL.

The prevailing response has been to treat schema quality as a fixed background condition and address its consequences *downstream*: constrained decoding improves syntactic validity but not naming ambiguity [39]; error-correction and self-refinement repair generated SQL after the fact [12, 31, 36], yet schema-linking errors rooted in opaque names persist; interactive clarification shifts the burden to users [14, 42, 51], impractical at scale; and recent ambiguity benchmarks probe query-level perturbations rather than schema-internal defects [3, 35, 38]. The schema itself—the root cause—remains untreated.

A small but growing line of work has begun to consider *upstream* schema-level interventions. Odin [43] recommends multiple SQL interpretations for ambiguous schemas at query time. CLEAR [53] provides a parser-independent disambiguation framework. While

these efforts represent important steps, they share two fundamental limitations. First, they operate at *query time*, incurring per-query overhead without producing durable schema-level improvements. Second, they lack a *grounded optimization objective*: the quality of a schema modification is assessed by linguistic plausibility rather than by its measurable effect on downstream task performance.

In this paper, we propose a fundamentally different approach. We observe that schema quality, viewed through the lens of Text-to-SQL, can be defined objectively as the downstream execution accuracy (ExAcc) it induces. This allows us to formalize schema refinement as a constrained optimization problem: find a renaming of schema elements that maximizes execution accuracy while preserving query equivalence via SQL views. We analyze the computational hardness—the search space is exponential (Observation 1) and the constrained problem is at least NP-hard (Theorem 1)—and propose EGRefine (Execution-Grounded Refinement), a four-phase pipeline whose key innovation is *execution-grounded verification*: rather than relying on an LLM's semantic judgment to select column names, we use downstream SQL execution results as the selection signal, treating execution accuracy as a reward function for schema refinement. This applies execution feedback at a novel granularity—not to the generated query as in prior work [8, 11, 40], but to the schema itself.

Our contributions are as follows:

(1) **Formal Problem Framing.** We cast Text-to-SQL schema refinement as a constrained optimization problem with a task-grounded objective (Definition 2), analyze its hardness (§3.3), and establish two structural properties: *column-local non-degradation* (Proposition 1) and *database-level query equivalence* (Proposition 2).
(2) **The EGRefine Framework.** We operationalize this as a four-phase pipeline—LLM-based screening (§4.1), candidate generation (§4.2), execution-grounded verification (§4.3), and non-destructive VIEW synthesis (§4.4)—that is system-agnostic at the interface: any downstream Text-to-SQL system can consume the refined schema without modification.
(3) **Comprehensive Empirical Evaluation.** We evaluate on three benchmarks: Dr.Spider [4] (controlled degradation), BIRD [26] (natural schemas), and BEAVER [10] (enterprise-scale stress test). Ablations show execution-grounded verification significantly outperforms pure LLM selection, regressions are rare, benefits amplify for weaker models, and a coverage–improvement relationship (§6) helps diagnose likely aggregate impact before deployment.

2 RELATED WORK

2.1 Text-to-SQL Systems

Text-to-SQL evolved from sequence models and schema-aware encoders [27, 48, 54], through constrained decoding and explicit schema linking [24, 39], to prompted and multi-agent LLM systems including DIN-SQL, DAIL-SQL, MAC-SQL, and CHESS [18, 33, 41, 47]; CodeS represents the fine-tuned line [25]. Even when large contexts reduce the need for explicit linking [30], all these methods consume a fixed schema. EGRefine is an orthogonal preprocessing layer that improves that input (see surveys [20, 22, 28, 50]).

2.2 Schema Ambiguity and Intervention

Spider-Syn, Spider-DK, and Dr.Spider expose brittleness under lexical, domain-knowledge, and schema perturbations [4, 16, 17]; realistic evaluations reach the same conclusion [15, 37]. Work on ambiguous or unanswerable questions instead modifies or clarifies the user input [3, 14, 34, 35, 38, 42, 46] and is complementary. Enterprise-oriented benchmarks provide descriptions or human annotations and increasingly large schemas [10, 23, 26, 49].

Odin, CLEAR, and adaptive abstention resolve schema uncertainty at query time [9, 43, 53]. Most closely, SNAILS [29] performs offline schema intervention: a classifier trained on human naturalness labels screens identifiers, and an LLM expands them into views. EGRefine instead optimizes ExAcc rather than linguistic naturalness, compares multiple names by execution, and retains the original when no gain exceeds $\tau_{\min}$. We compare directly with the released SNAILS screening signal and with an LLM-Direct baseline under our own screening (Table 1).

2.3 Execution-Guided Optimization

Execution feedback constrains, reranks, or repairs SQL [5, 12, 31, 39, 47] and supports verification and debugging in general code generation [8, 11, 40]. These methods optimize a generated query while holding its schema fixed; EGRefine optimizes the input schema once for subsequent queries. The design also parallels workload-driven physical design for indexes, materialized views, and database knobs [2, 6, 7, 32, 44, 45], but tunes the logical naming layer through an execution-grounded what-if interface.

3 PROBLEM FORMULATION

3.1 Preliminaries and Notation

Schema. Let $\mathcal{S} = (\mathcal{T}, \mathcal{C}, \mathcal{F})$ be a relational schema. Its table set is $\mathcal{T} = \{t_1, \ldots, t_n\}$, and its column set is $\mathcal{C} = \{c_1, \ldots, c_m\}$. The set $\mathcal{F} \subseteq \mathcal{C} \times \mathcal{C}$ encodes foreign-key relationships. Each column c belongs to table $\mathrm{tab}(c)$ and has visible surface name $\mathrm{name}(c)$. Its naming scope is

$$\mathrm{scope}(c) = \{c' \in \mathcal{C} \mid \mathrm{tab}(c') = \mathrm{tab}(c)\} \cup \{c' \mid (c, c') \in \mathcal{F} \text{ or } (c', c) \in \mathcal{F}\}. \quad (1)$$

covering same-table and FK-related columns; unrelated cross-table homonyms remain admissible.

Task. Model M maps question nl and schema $\mathcal{S}$ to SQL. For benchmark $Q = \{(nl_j, sql_j^*)\}_{j=1}^{|Q|}$, execution accuracy is

$$\mathrm{ExAcc}(M, \mathcal{S}, Q) = \frac{1}{|Q|} \sum_{j=1}^{|Q|} \mathbb{1}\big[\mathrm{exec}(M(nl_j, \mathcal{S})) = \mathrm{exec}(sql_j^*)\big], \quad (2)$$

where exec returns the query result.

3.2 Schema Refinement as Optimization

Definition 1 (Refinement Mapping). *A* refinement mapping $r : \mathcal{C} \to \Sigma^*$ *assigns a (possibly new) surface name* $r(c)$ *to each column* $c \in \mathcal{C}$*, where* Σ^* *is the set of valid SQL identifiers. The identity mapping* r_{id} *corresponds to the unchanged schema; we write* $r(\mathcal{S})$ *for the schema obtained by applying* r*. A refinement is* admissible

if (i) no two columns in the same scope receive identical names, and (ii) only surface names are modified (data types, constraints, foreign keys, and table structure remain unchanged). We denote the set of admissible refinements by $\mathcal{R}(\mathcal{S})$.

Definition 2 (Schema Quality). *Given $\mathcal{S}$, a set of Text-to-SQL models $\mathcal{M} = \{M_1, \ldots, M_l\}$, and a query set Q:*

$$\text{Quality}(\mathcal{S}, \mathcal{M}, Q) = \frac{1}{|\mathcal{M}|} \sum_{M_j \in \mathcal{M}} \text{ExAcc}(M_j, \mathcal{S}, Q). \tag{3}$$

Problem 1 (Optimal Schema Refinement). *Given $\mathcal{S}$, $\mathcal{M}$, and Q, find:*

$$r^* = \arg\max_{r \in \mathcal{R}(\mathcal{S})} \text{Quality}\big(r(\mathcal{S}), \mathcal{M}, Q\big), \tag{4}$$

subject to:

$$\text{exec}(q', r(\mathcal{S})) = \text{exec}(q, \mathcal{S}),$$

for every query q' over $r(\mathcal{S})$ and its semantically equivalent query q over $\mathcal{S}$.

View materialization satisfies this non-destructive constraint by construction (Proposition 2).

Remark 1. *A column c is* refinement-improvable *iff some alternative name c' strictly increases* Quality *when it replaces* name(c).

3.3 Hardness Analysis

Observation 1 (Exponential Search Space). *Let $|C| = m$ and suppose each column has k candidates (including the original). Then $|\mathcal{R}(\mathcal{S})| = O(k^m)$: for m=100 and k=3, $|\mathcal{R}(\mathcal{S})| > 5 \times 10^{47}$, rendering exhaustive search infeasible.*

Definition 3 (Constrained-Refinement-Decision (CRD)). *Given a schema $\mathcal{S}$ with column set $C = \{c_1, \ldots, c_n\}$, within-scope conflict relation $E \subseteq C \times C$, candidate lists $\mathcal{L}(c_i) \subseteq \Sigma^+$ ($|\mathcal{L}(c_i)| \geq 1$), and a forced-rename subset $C^\dagger \subseteq C$, does there exist an assignment $r : C \to \Sigma^+$ satisfying (i) $r(c_i) \in \mathcal{L}(c_i)$ for all i; (ii) $r(c_i) \neq r(c_j)$ for all $(c_i, c_j) \in E$; and (iii) $r(c_i) \neq c_i^0$ for all $c_i \in C^\dagger$?*

Theorem 1 (NP-hardness). *The CRD problem is NP-hard. The optimization version of Problem 1—maximizing* Quality *over assignments satisfying the same constraints—is therefore also NP-hard.*

Proof. Reduce from List-Coloring [21]. Map each vertex v_i to a column c_i in one scope, set $E = E_G$, and encode its color list as $\mathcal{L}(c_i) = \{\sigma(\ell) : \ell \in L(v_i)\}$ for injective σ; set $C^\dagger = \emptyset$. Then $r(c_i) = \sigma(\phi(v_i))$ is a CRD assignment iff ϕ is a proper list coloring. The polynomial reduction proves CRD, and therefore optimization with the same constraints, NP-hard. □

3.4 Decomposition and Structural Properties

We therefore replace joint search with independent per-column subproblems.

3.4.1 Element-Wise Refinement. For each column $c_i \in C$, let $Q(c_i) = \{(nl, sql^*) \in Q \mid c_i \text{ appears in } sql^*\}$ denote the queries referencing c_i. For each candidate column, score the original and each proposal on $Q(c_i)$:

$$s_i(c') = \text{Quality}\big(\mathcal{S}[c_i \to c'], \mathcal{M}, Q(c_i)\big), \qquad c' \in \text{Cand}(c_i) \cup \{c_i^0\}, \tag{5}$$

where c_i^0 is the original, $\text{Cand}(c_i)$ contains the k proposals, and $\mathcal{S}[c_i \to c']$ applies only that substitution.

3.4.2 Conservative Selection Rule. Let

$$\Delta_i = \max_{c' \in \text{Cand}(c_i)} s_i(c') \; - \; s_i(c_i^0) \tag{6}$$

denote the quality gain of the best candidate over the original name. The conservative selection is:

$$c_i^* = \begin{cases} \arg\max_{c' \in \text{Cand}(c_i)} s_i(c'), & \text{if } \Delta_i \geq \tau_{\min}; \\ c_i^0, & \text{otherwise,} \end{cases} \tag{7}$$

where $\tau_{\min}$=0.05 suppresses marginal gains (sensitivity in Table 4).

Proposition 1 (Per-Column Local Non-Degradation). *Holding all other columns fixed, the conservative rule (7) never decreases quality on the column-local query subset:*

$$\text{Quality}\big(\mathcal{S}[c_i \to c_i^*], \mathcal{M}, Q(c_i)\big) \;\geq\; \text{Quality}\big(\mathcal{S}, \mathcal{M}, Q(c_i)\big). \tag{8}$$

Proof. Immediate from (7): a new name is adopted only when $\Delta_i \geq \tau_{\min} \geq 0$, so $s_i(c_i^*) = s_i(c_i^0) + \Delta_i \geq s_i(c_i^0)$; otherwise the original name is kept and the two sides of (8) coincide. □

Remark 2 (Scope of the Guarantee). *The guarantee holds only on $Q(c_i)$ with other columns fixed. Full-schema prompts can affect non-touching queries, and jointly deployed renames can interact; hence EGRefine is empirically regression-resistant, not globally non-degrading. In the composed audit (§6.10), 30 of 49 replicated gains remain positive, 14 become neutral, and five become negative; reverting the five produces no detectable end-to-end difference.*

3.4.3 Global Conflict Resolution. For a within-scope collision, we keep the higher-Δ rename and assign the other its next-best non-conflicting candidate; after two rounds, unresolved columns revert to their originals.

3.4.4 Query Equivalence via VIEW Synthesis. The refined schema $r(\mathcal{S})$ is materialized as a set of SQL `CREATE VIEW` statements aliasing original column names to their refined counterparts:

```
CREATE VIEW t̂ AS SELECT
    c_1 AS r(c_1), ..., c_n AS r(c_n) FROM t,
```
(9)

for each table t with columns $c_1, \ldots, c_n$. The DBMS resolves refined identifiers through the views; no string rewriting is required.

Proposition 2 (Query Equivalence under DBMS View Semantics). *Let r be an admissible refinement mapping, and let $V = \text{VIEW}(r, \mathcal{S})$ be the corresponding view definitions. For any read-only SQL query q' over $r(\mathcal{S})$ consisting of relational-algebra core operators (projection, selection, equi-join, set operations) over the views, there exists a semantically equivalent query q over $\mathcal{S}$ such that:*

$$\text{exec}(q, \mathcal{S}) = \text{exec}(q', r(\mathcal{S})). \tag{10}$$

Algorithm 1 EGRefine Pipeline

Require: Schema $\mathcal{S}$, models $\mathcal{M}$, queries Q, data samples D
Ensure: Refined schema $\mathcal{S}^*$ (as SQL VIEWs)

$A \leftarrow$ Phase1-Screen$(\mathcal{S}, \mathcal{M}_{\text{screen}})$ ▷ LLM screening
$A \leftarrow A \setminus$ StructuralExclude$(\mathcal{S})$ ▷ Remove FK / join keys
for each column $c_i \in A$ **do**
  Cand$(c_i) \leftarrow$ Phase2-Generate$(c_i, \mathcal{S}, D)$
  $c_i^*, \Delta_i \leftarrow$ Phase3-Verify$(c_i, \text{Cand}(c_i), Q, \mathcal{M})$
end for
$R \leftarrow \{(c_i, c_i^*) \mid \Delta_i \geq \tau_{\min}\}$
$R \leftarrow R \cup$ PropagatePK$(R, \mathcal{S})$ ▷ PK→FK
$\mathcal{S}^* \leftarrow$ Phase4-Synthesize$(R, \mathcal{S})$ ▷ CREATE VIEW
return $\mathcal{S}^*$

For SQL features beyond the core algebra (aggregation, `ORDER BY`*,* `LIMIT`*,* `DISTINCT`*, scalar functions, quoted identifiers,* `NATURAL JOIN`*), equivalence is provided by DBMS view-expansion semantics [19].*

Proof. Equation (9) is a rename ρ_r. Rename commutes with the core operators [1], and admissibility makes r^{-1} well-defined within scope, so e' rewrites to $e'[r^{-1}]$. Other listed SQL constructs follow DBMS view-expansion semantics [19]; our three benchmarks exhibit no execution discrepancy between original and aliased gold SQL. □

4 METHOD: EGREFINE

EGRefine implements the decomposition in four phases (Figure 1; Algorithm 1).

Deployment setting. This offline step consumes a representative question–gold-SQL workload, analogous to workload-driven physical design [2, 6], and amortizes refinement over future queries and models. It should be rerun after drift; §5.7 tests unseen queries.

4.1 Phase 1: LLM-Based Schema Screening

Phase 1 asks an LLM whether each name could confuse Text-to-SQL, given the domain, table/type, neighboring columns, five rows, and criteria for abbreviation, polysemy, codes, and generic vocabulary. It returns $A \subset C$; Phase 3 absorbs false positives.

Structural exclusion. Primary keys remain eligible, while all foreign keys and same-name/type cross-table columns are excluded as explicit or implicit join keys.

4.2 Phase 2: Context-Aware Candidate Generation

Phase 2 generates k=3 ranked names from the domain, neighboring columns/types/values, 20 target values, and conservative guidelines, forming $\text{Cand}^+(c_i) = \{c_i^0\} \cup \text{Cand}(c_i)$. Every call sees the same unmodified schema before Phase 3, preventing order feedback; composition risk arises only when isolation-scored renames are deployed jointly (§6.10).

4.3 Phase 3: Execution-Grounded Verification

Phase 3 replaces the LLM ranking with execution accuracy on the column's verifier queries.

4.3.1 Query Subset Selection. Set $Q(c_i) = \{(nl, sql^*) \in Q : c_i \in \text{cols}(sql^*)\}$; if empty, retain the original.

4.3.2 Per-Candidate Scoring. For each name, create a temporary view, generate SQL for $Q(c_i)$, and compare predicted and gold execution:

$$\text{Score}(c_j') = \text{ExAcc}(M, \mathcal{S}[c_i \to c_j'], Q(c_i)). \tag{11}$$

4.3.3 Multi-Algorithm Aggregation. To reduce dependence on any single Text-to-SQL system, we aggregate scores:

$$\overline{\text{Score}}(c_j') = \frac{1}{|\mathcal{M}|} \sum_{M \in \mathcal{M}} \text{Score}_M(c_j'). \tag{12}$$

We verify with $\mathcal{M}$ = {C3, DIN-SQL} and hold out MAC-SQL; the M=1 ablation tests dependence on one verifier.

4.3.4 Conservative Selection. The best candidate is selected as:

$$c_i^* = \arg \max_{c_j' \in \text{Cand}^+(c_i)} \overline{\text{Score}}(c_j'), \tag{13}$$

subject to:

$$\Delta_i = \overline{\text{Score}}(c_i^*) - \overline{\text{Score}}(c_i^0) \geq \tau_{\min}, \tag{14}$$

where $\tau_{\min}$ is from Eq. (7); otherwise the original is retained, instantiating Proposition 1.

4.3.5 Computational Cost. Cost is $O(|A|k|\mathcal{M}|\overline{|Q(c)|})$. The measured k=3 run performs 112,449 comparisons and 224,898 SQL executions. Phase 3 accounts for 97.0% of the 35.2 accumulated database-hours. With eight workers on three endpoints, the run finishes in about 5.2 wall-clock hours (Table 2). Request counts for $k = 1, 2, 3, 5$ are 58.4k, 85.9k, 114.7k, and 159.6k. At $k = 2$, calls fall 25.1% from $k = 3$, with no detectable accuracy difference; $k = 3$ commits 80 rather than 54 renames. We retain $k = 3$ only to keep the main experiments comparable. For a cost-sensitive deployment, $k = 2$ is the better choice. Reducing sampled rows from 20 to 5 also shows no loss. This offline cost is amortized over future queries and downstream models. We omit a monetary estimate because token usage was not logged.

4.4 Phase 4: Non-Destructive Schema Synthesis

After propagation and conflict resolution, Phase 4 enforces Definition 1's admissibility constraints as a hard post-condition, then materializes Eq. (9) as read-only views over the original tables. Deployment still handles dialect, quoting, optimizer, and access-control details.

4.4.1 PK→FK Name Propagation. For renamed PK $(T_p, c_p) \to c_p'$, propagate to a referencing FK only when its surface name equals c_p case-insensitively: the same opaque name shares the ambiguity. Role-named FKs such as statement_account retain their local semantics and remain outside independent screening. Propagation is deterministic rather than separately verified; §6.7 audits it. Empty-$Q(c_i)$ columns remain original, and residual collisions revert the lower-Δ rename.

5 EXPERIMENTS

We test recovery (RQ1), components and sensitivity (RQ2), cross-model transfer (RQ3), and real-schema deployment (RQ4).

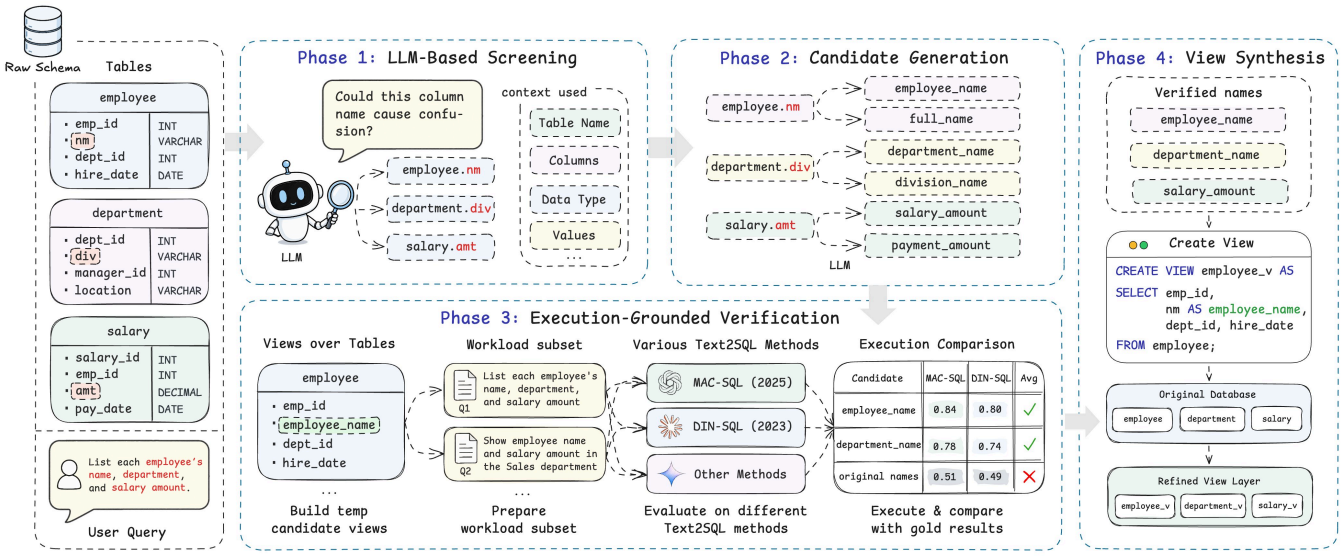


Figure 1: EGRefine: screen columns, generate names, verify them by execution, and synthesize non-destructive views. Only Phase 3 consumes the user workload.

5.1 Experimental Setup

5.1.1 Benchmarks. Dr.Spider [4] provides two controlled Spider [52] variants: Schema-Abbreviation (2,853 queries, 691 perturbed columns, 90 databases) and Column-Synonym. BIRD [26] offers 1,534 hint-free queries across 11 natural schemas. BEAVER [10] has 88 enterprise queries across five databases (163–1,269 columns); four are refined (1,439 columns).

5.1.2 Text-to-SQL Systems. We evaluate zero-shot C3 [13], decomposed DIN-SQL [33], and MAC-SQL-style multi-agent prompting [47] under one harness. Qwen3.5-27B is default; RQ3 adds 9B, Gemma3-27B, and Phi-4-14B.

5.1.3 Refinement Baselines. Baselines are NoRef; LLM-Direct (top-1 after Phases 1–2); SNAILS screening [29] followed by unverified Qwen renaming; and LLM-generated description comments (§5.8).

5.1.4 Metrics. We report Execution Accuracy (ExAcc). For Dr.Spider, we additionally report *Recovery Rate*:

$$\text{RecRate} = \frac{\text{ExAcc}_{\text{refined}} - \text{ExAcc}_{\text{degraded}}}{\text{ExAcc}_{\text{clean}} - \text{ExAcc}_{\text{degraded}}} \times 100\% \qquad (15)$$

where values above 100% exceed the clean Spider schema.

5.1.5 Implementation. Qwen3.5-27B runs locally with $k = 3$, $N_s = 20$, $\tau_{\text{min}} = 0.05$, and $\mathcal{M} = \{\text{C3, DIN}\}$; MAC is out-of-loop. The headline run used the submitted unconditional PK–FK policy; the revised same-name rule affects five databases and shows no paired loss (§6.7), so cross-endpoint absolute scores are not spliced into Table 1.

5.2 RQ1: Recovery from Schema Degradation

Table 1 shows +2.56/+2.48/+0.63 pp and 134.7/67.4/15.0% recovery on C3/DIN/MAC. SNAILS degrades all three and LLM-Direct harms DIN/MAC; EGRefine beats SNAILS by 6.1/5.9/2.1 pp ($p < 0.001$). On Column-Synonym it adds 2.25 pp, extending beyond abbreviations.

Table 1: Execution Accuracy (%) on Dr.Spider Schema-Abbreviation (2,853 queries, Qwen3.5-27B). Recovery Rate per Eq. 15. †SNAILS [29] uses its released naturalness classifier to screen columns, which are then renamed by Qwen3.5-27B and accepted without execution verification, isolating the selection signal (§2.2).

Method	C3	DIN-SQL	MAC-SQL
Clean (Original Spider)	72.77	80.41	63.79
Degraded (Abbreviated)	70.87	76.73	59.59
+ LLM-Direct	72.34	75.85	57.17
+ SNAILS†	67.33	73.33	58.15
+ **EGRefine**	**73.43**	**79.21**	**60.22**
Δ (EGRefine − Degraded)	+2.56	+2.48	+0.63
Recovery Rate	134.7%	67.4%	15.0%

5.3 RQ2: Component Ablation and Sensitivity

Table 3 isolates each component.

Removing execution is worst and pushes DIN/MAC below NoRef; removing the margin is milder, so execution is primary and abstention secondary. Removing screening lowers C3/DIN by 2.35/1.89 pp. C3-only verification preserves C3 but harms held-out DIN/MAC, motivating multi-system consensus.

Table 4 shows that 0.01 over-commits and regresses DIN/MAC, while 0.10 suppresses C3/DIN gains; 0.05 is best on all three.

Table 2: Offline cost of the measured $k = 3$ Dr.Spider-Abbr rebaseline. Time is accumulated per database, not wall-clock. Counts are logical requests/comparisons; transport retries and token usage were not logged.

Phase	Operation	LLM requests	Exec. comparisons	Accumulated time
1	Ambiguity screening (1,605 columns)	1,605	0	5.7 min
2	Candidate generation (665 columns)	665	0	56.8 min
3	Execution-grounded verification	112,449	112,449	34.2 h
4	VIEW synthesis	0	0	seconds
Total		114,719	112,449	35.2 h

Table 3: Ablation on Dr.Spider Schema-Abbreviation (ExAcc %). M=1 uses C3-only verification; full pipeline uses M=2 (C3+DIN-SQL).

Variant	C3	DIN-SQL	MAC-SQL
EGRefine (full, M=2)	**73.43**	**79.21**	**60.22**
w/o Execution	72.34	75.85	57.17
w/o Conservative	73.01	78.83	58.92
w/o Screening	71.08	77.32	58.78
w/o Multi-Algo (M=1)	73.36	76.34	59.48
Degraded (no refine)	70.87	76.73	59.59

Table 4: Threshold sensitivity on Dr.Spider-Abbr; Δ is vs. NoRef.

$\tau_{\min}$	Committed	ΔC3	ΔDIN-SQL	ΔMAC-SQL
0.01	104	+0.70	−0.25	−0.81
0.03	89	+0.81	+1.15	−0.11
0.05	**81**	**+2.56**	**+2.48**	**+0.63**
0.10	54	+0.70	+0.94	+0.56

Table 5: Cross-verifier robustness on Dr.Spider Schema-Abbreviation (Qwen3.5-27B). Original $\mathcal{M}$ = {C3, DIN}; alternative $\mathcal{M}'$ = {DIN, MAC}. Held-out role is shaded.

Verifier set	Algo	ExAcc (%)	Δ	Recovery
{C3, DIN} (original)	C3	73.43	+2.56	134.7%
	DIN-SQL	79.21	+2.48	67.4%
	MAC-SQL	60.22	+0.63	15.0%
{DIN, MAC} (alternative)	C3	**75.34**	**+4.47**	**235.3%**
	DIN-SQL	79.87	+3.14	85.2%
	MAC-SQL	63.17	+3.58	85.2%

Changing verifiers to {DIN, MAC} keeps every algorithm positive (+0.63 to +4.47 pp; Table 5); held-out C3 is strongest, while MAC remains bounded even in-loop (§6.4).

Candidate count k and sampled rows N_s. Table 6 shows a plateau from $k = 2$: it uses 25.1% fewer calls than $k = 3$, with no C3/DIN difference ($p = 1.00/0.28$). The two settings commit 54 and 80 names, respectively. We retain the pre-fixed $k = 3$ for comparability, not optimality; $k = 2$ is budget-efficient. $k = 5$ costs 39% more without gain, and $N_s = 5$ matches 20.

Table 6: k and sample-row ablation on the replication endpoint. NoRef is 70.10/76.45; every arm beats it at $p < 0.001$.

Setting	Commits	LLM calls	C3	DIN-SQL
k=1	53	58,359	72.24	78.79
k=2	54	85,924	71.99	79.14
k=3 (default)	80	114,719	71.96	**79.50**
k=5	96	159,644	72.38	79.36
N_s=5 (default 20)	80	108,609	72.38	79.39

Table 7: Refine × Eval transfer matrix on Dr.Spider Schema-Abbreviation. Δ = improvement over eval model's no-refinement baseline.

Refine	Eval	C3 (Δ)	DIN (Δ)	MAC (Δ)
9B	9B	63.34 (+3.96)	75.81 (+5.18)	53.63 (+2.11)
27B	9B	62.36 (+2.98)	75.60 (+4.97)	53.49 (+1.97)
9B	27B	73.22 (+2.35)	**79.53** (+2.80)	59.34 (−0.25)
27B	27B	**73.43** (+2.56)	79.21 (+2.48)	**60.22** (+0.63)

5.4 RQ3: Cross-Model Transferability

A practical question is whether a refined schema produced by one model benefits a different model at serving time, enabling *refine-once, serve-many-models* deployment.

5.4.1 Multi-Backbone Evaluation. Table 8 evaluates three serving backbones. The shared 27B-refined schema improves every system, with DIN-SQL recovery exceeding 100% on Gemma3-27B (133.7%) and Phi-4 (188.1%).

5.4.2 Transfer Matrix. Tables 7 and 8 show that strong→weak transfer stays within 1 pp of 9B self-refinement. Weak→strong transfer also remains viable: 9B-refined DIN scores 79.53 vs. 79.21 when self-refined. Phi-4 loses 0.63 pp when self-refining but gains 0.74 pp from 27B refinement, making candidate generation—not serving—the weak-model bottleneck.

Table 8: Self-refinement vs. transferred refinement across serving backbones (Dr.Spider-Abbr, 2,853 queries, ExAcc %). "27B-refined" serves the Qwen3.5-27B-refined schema to each backbone at no re-verification cost. The weakest backbone (Phi-4) loses from self-refinement; 9B self-refines well, and the transferred schema stays within 1 pp of self-refinement on every system.

Backbone	Schema	C3	DIN-SQL	MAC-SQL
Qwen3.5-9B	No refinement	59.38	70.63	51.52
	self-refined	**63.34**	**75.81**	**53.63**
	27B-refined	62.36	75.60	53.49
Phi-4 14B	No refinement	56.85	69.54	34.00
	self-refined	56.22	69.93	32.32
	27B-refined	**57.59**	**72.38**	**34.38**
Gemma3-27B	No refinement	69.33	75.67	34.77
	27B-refined	**71.15**	**76.94**	**35.02**

Table 9: Execution verification necessity on BIRD (Qwen3.5-9B, 1,534 queries, no hints). LLM-Direct = verification-free LLM baseline.

Method	C3	DIN-SQL	MAC-SQL
No Refinement	28.75	31.75	20.60
+ LLM-Direct	28.23 ↓0.52	30.12 ↓1.63	19.17 ↓1.43
+ **EGRefine**	**30.25** ↑1.50	**33.25** ↑1.50	20.14 ↓0.46
Separation (EGR−Dir)	+2.02	+3.13	+0.97

Table 10: BIRD evidence × refinement ablation (Qwen3.5-27B, 1,534 queries). Evidence and EGRefine are complementary.

Configuration	C3	DIN	MAC
No Refinement	41.53	38.59	29.99
+ Evidence	54.24	62.13	46.74
+ EGRefine	41.72	39.24	31.23
+ Both	**54.82**	**63.56**	**47.33**
Δ (Both − Evidence)	+0.58	+1.43	+0.59

5.5 RQ4: Deployment on Real-World Schemas

While Dr.Spider uses controlled perturbations, real-world schemas exhibit naturally occurring naming issues. We validate on BIRD (curated academic schemas) and BEAVER (enterprise data warehouses); Table 11 gives the NoRef–EGRefine overview for both.

5.5.1 BIRD: Execution Verification is Essential. On BIRD's professionally curated schemas, EGRefine conservatively refines only 28 of 798 columns (3.5%), yielding modest but positive aggregate improvements on 27B (+0.19/+0.65/+1.24 on C3/DIN/MAC). A finer-grained analysis in §6.6 reveals an algorithm-specific differential: on the 126-query touching subset, DIN-SQL gains +6.35 pp while C3 (−1.59 pp) and MAC-SQL (−3.17 pp) do not benefit—a pattern we trace to agent-level brittleness (§6.4).

Table 11: Real-world deployment results (ExAcc %). BIRD: full 1,534-query dev set. BEAVER: fair same-database comparison on the four databases covered by refinement (45 queries; §5.5.2); Qwen3.5-27B commits no renames on BEAVER, so its refined row equals NoRef by construction; for the BEAVER rows, the parenthesized model is the refinement backbone and the serving systems are unchanged.

Benchmark (backbone)	Schema	C3	DIN-SQL	MAC-SQL
BIRD (27B)	No refinement	41.53	38.59	29.99
	+ EGRefine	**41.72**	**39.24**	**31.23**
BIRD (9B)	No refinement	28.75	31.75	20.60
	+ EGRefine	**30.25**	**33.25**	20.14
BEAVER (27B)	No refinement	13.33	13.33	6.67
	+ EGRefine (0 commits)	13.33	13.33	6.67
BEAVER (MiniMax-M2.7)	No refinement	13.33	13.33	6.67
	+ EGRefine	13.33	13.33	**11.11**

Robustness of the BIRD aggregate effect. Paired bootstrap 95% CIs show that most per-cell BIRD gains are not individually significant. For example, 27B DIN-SQL gains +0.65 (CI [−0.5, +1.8]); the strongest cell, 9B DIN-SQL, gains +1.50 (CI [+0.07, +2.9]). This uncertainty is consistent with BIRD's low refinement coverage (3.5% of columns, §6.1). We therefore base the BIRD claim on several results: six of seven cells are positive, and the workload holdout (§5.7), touching subset (§6.6), 9B reproduction (Table 9), and coverage analysis (§6.1) agree in direction and magnitude. On Dr.Spider, where coverage is much higher, the C3 and DIN-SQL gains are individually significant (§6.3).

The critical finding emerges on the weaker 9B backbone (Table 9): LLM-Direct *degrades* all three systems (−0.52/−1.63/−1.43), while EGRefine improves C3 and DIN-SQL (+1.50/+1.50) with marginal regression on MAC-SQL (−0.46). The 0.97–3.13 pp separation between verified and unverified refinement establishes execution-grounded verification as necessary for safe refinement on smaller models.

5.5.2 BEAVER: Enterprise-Scale Applicability. BEAVER contains 88 queries over five enterprise databases with 163–1,269 columns. Our matched refinement comparison uses four of them, totaling 1,439 columns. The 1,269-column database exceeds the refinement budget and is excluded from both arms. On the four included databases, NoRef scores 13.33% on C3 and DIN-SQL, and 6.67% on MAC-SQL. Across all five databases, baseline ExAcc is lower still (7.95%/7.95%/3.41%). In this regime, residual errors are dominated by schema assembly and query generation rather than identifier interpretation, leaving few execution-score differences among candidate names.

Conservative selection. With Qwen3.5-27B as refiner, Phase 3 rejects all 106 Phase 1 candidates—the conservative rule operating as designed when Δ scores cannot be reliably distinguished from noise. Testing whether refiner capability is the bottleneck, we repeat with MiniMax-M2.7: Phase 1 screens 172 candidates (vs. Qwen's 106) and Phase 3 commits 4 refinements (vs. 0). Despite this, downstream ExAcc on C3 and DIN-SQL is unchanged (13.33%→13.33%

on the four databases covered by refinement; +0.00 pp): Phase 3 in fact screened and execution-scored the columns these queries reference (e.g., `group.extra`, `user.extra`), yet no candidate rename changed ExAcc (Δ=0), so the conservative rule kept them. On BEAVER's hardest schemas, naming is simply not the accuracy bottleneck for the outcome-determining columns—the invariance is a downstream ceiling, not a verification failure (cf. §6.5). The lone exception is MAC-SQL: 6.67%→11.11% (3/45→5/45) on the same four databases, a gain of 4.44 pp. This reverses the Dr.Spider pattern, where MAC benefits least. On BEAVER's harder queries, MAC-SQL rarely reaches the aggressive re-decomposition stage that causes regressions on easier benchmarks. The result is therefore consistent with the agent-instability analysis in §6.4. §6.11 analyzes this boundary in detail. Across the three systems, 96.3% of the 242 residual errors do not assemble the gold column set. A separate name-sensitivity probe finds that 96% of verified columns are insensitive to the tested candidates. Together, these diagnostics localize the dominant bottleneck to retrieval and assembly under the tested refiners.

5.6 Complementarity with Domain Knowledge

BIRD provides optional per-query *evidence* (domain hints). A 2×2 ablation (Table 10) shows that EGRefine adds +0.58/+1.43/+0.59 pp on top of evidence for C3/DIN/MAC. The two signals serve different roles: evidence explains a column in a specific query, whereas refinement changes the identifier seen across queries.

5.7 Workload-Holdout Validation

Dev-split validation on real queries. Our primary holdout evidence splits each benchmark's real queries into a refinement part and a held-out evaluation part (Table 12). On Dr.Spider-Abbr, the 50/50 split provides the strongest result: DIN-SQL reaches 78.88 ExAcc, 3.01 points above NoRef (McNemar $p<0.001$). The remaining split-vs.-NoRef differences are small. The 70/30 gains are +0.94 for C3 and +0.36 for DIN-SQL, but neither is significant. At 50/50, C3 gains +0.70 (p=0.343). BIRD changes by −0.26/+0.00 (p=0.888/1.000).

Table 12 reports the canonical-vs.-split comparison separately. At 70/30, the split and full-workload schemas are statistically indistinguishable (C3 p=1.000; DIN-SQL p=0.146). At 50/50, the full-workload schema leads on C3 (+1.40, p=0.029) but not on DIN-SQL (p=0.711). This comparison changes both query visibility and workload size: the two schemas commit 81 and 58 renames. The workload-size ablation in Table 13 is positive at every tested size and peaks at 100%, so the C3 difference is better explained by workload size than by a consistent query-specific effect. BIRD's sub-point changes provide no contrary pattern.

A stricter forced-target test conditions candidate generation on queries (Table 18). At the default temperature, the conditioned and standard arms tie on C3 and DIN-SQL. A deterministic sensitivity run changes one C3 outcome and again ties on DIN-SQL (McNemar p=1). The detailed analysis in §6.9 supports retaining the simpler query-blind generator with execution-grounded selection.

Independent-authoring synthetic probe. As a complementary, stronger-independence but lower-power test, we retain the submitted BIRD holdout produced by a different authoring process.

Table 12: Dev-split validation on unseen test queries (ExAcc, %). Paired entries are C3/DIN-SQL. $\Delta_{s:n}$ and $p_{s:n}$ compare split-refined with NoRef; $p_{c:s}$ compares canonical (full-Q) with split-refined (exact two-sided McNemar). Dr.Spider-Abbr splits by base-question pool (seed 42); BIRD uses a stratified 50/50 split; test sizes are 851/1,430/770, respectively.

Benchmark	Schema	ExAcc	$\Delta_{s:n}$	$p_{s:n}$	$p_{c:s}$
ABBR 70/30	NoRef	69.68/75.32	–	–	–
	split	70.62/75.68	+0.94/+0.36	0.302/0.629	–
	canonical	70.51/76.38	–	–	1.000/0.146
ABBR 50/50	NoRef	71.96/75.87	–	–	–
	split	72.66/78.88	+0.70/+3.01	0.343/<0.001	–
	canonical	74.06/78.67	–	–	0.029/0.711
BIRD 50/50	NoRef	41.17/39.22	–	–	–
	split	40.91/39.22	−0.26/+0.00	0.888/1.000	–
	canonical	41.56/38.96	–	–	0.590/0.860

Table 13: Workload-size ablation (Dr.Spider-Abbr, evaluated on the full benchmark; revision replication endpoint — within-arm comparisons only). Refinement uses nested workload subsets (seed 42) with shared Phase-2 candidates; only the per-column query sets $Q(c_i)$ vary. Δ = vs. NoRef (70.10/76.45 on this endpoint). Gains are positive at every tested size and largest at 100%; at 100%, McNemar $p<0.001$ (net +66/+88). Commit counts are not nested across arms — each arm re-estimates Δ on its own $Q(c_i)$, so committed sets differ.

Workload $\lvert Q\rvert$	Commits	ΔC3	ΔDIN-SQL
715 (25%)	34	+1.05	+1.61
1,433 (50%)	70	+1.58	+2.91
2,136 (75%)	66	+1.54	+2.03
2,853 (100%)	73	**+2.32**	**+3.08**

DeepSeek-V4-Pro authored 110 questions from official column descriptions while real identifiers were hidden behind `col_1`, `col_2`, …; it saw the names only after each question was fixed, to author gold SQL. Execution validation retained 96 non-duplicate, non-empty queries (65 aggregations, 37 `GROUP BY`, 32 `ORDER BY`, and 22 subqueries; mean SQL length 184 characters and mean JOIN count 0.93).

On these 96 unseen queries, the fixed refined schema scores 48.96 versus 47.92 NoRef (+1.04 pp; 95% CI [−3.1, +5.2]). This smaller, independent-authoring probe agrees in direction with the real-query split and complements the larger DIN-SQL holdout test.

Out-of-distribution generalization. We distinguish three shifts. (i) *In-distribution unseen queries*: the dev splits above use the same authoring process. DIN-SQL on the Dr.Spider-Abbr 50/50 split provides strong transfer evidence; the other cells provide smaller estimates. The synthetic holdout adds a different authoring process. (ii) *Vocabulary shift*: Dr.Spider-Syn uses synonym substitutions rather than abbreviations. EGRefine gains +2.25 pp on C3 (§5.2), so the result is not specific to one perturbation family. (iii) *Workload drift*: no formal guarantee extends beyond the verification workload (Remark 2). The unseen-query and composition audits provide

Table 14: Identifier replacement vs. description annotation (C3 + Qwen3.5-27B, n=2853). Description annotations are LLM-generated SQL comments. ExAcc differs slightly from Table 1 due to prompt-formatting tokenization effects.

Schema	Description	ExAcc	Δ vs. NoRef
NoRef	no	70.83	—
NoRef	yes	72.48	+1.65
EGRefine	no	73.46	+2.63
EGRefine	yes	74.69	**+3.86**

empirical checks, not a uniform guarantee (§6.10). VIEW equivalence (Proposition 2) preserves the original database semantics and allows rollback with a single `DROP VIEW`.

After drift, the pipeline can be rerun at the measured offline cost in §4.3.5. In the reported Dr.Spider-Abbr configuration, the smallest tested workload (715 queries) already gives positive aggregate gains (Table 13). This is an observed operating point, not a universal threshold, because support varies by column through $|Q(c_i)|$.

5.8 Identifier Replacement vs. Description Annotation

A natural alternative to renaming columns is to leave column identifiers untouched and inject column descriptions as SQL comments into the schema prompt. We test whether this non-invasive route is sufficient.

Protocol. We reuse EGRefine's Phase 1 screened column set on Dr.Spider-Abbr (665 columns) and prompt Qwen3.5-27B to generate one 8–25-word SQL comment per column from its name, neighbor columns, and 20 sampled values. The descriptions are injected as inline SQL comments (e.g., `gf TEXT, -- The country's form of government, e.g., Republic, Monarchy`); column identifiers are unchanged. Critically, the description baseline performs no execution verification—a single-pass annotation matching the simplest deployment without our full pipeline. We evaluate four cells with C3 + Qwen3.5-27B (Table 14).

Result. Description annotation alone yields +1.65 pp—non-trivially helpful, ruling out a strawman baseline. Identifier replacement (EGRefine) yields +2.63 pp, exceeding description annotation by 0.98 pp on the same Phase 1 column set and same backbone. The combined treatment yields +3.86 pp, 0.42 pp below the linear sum of the two—indicating partial redundancy: once a column is renamed to a semantically clear identifier, an additional description provides diminishing returns.

Scope of the comparison. The two arms represent practical deployment configurations rather than isolating representation alone: EGRefine uses execution-grounded selection, while the description arm uses single-pass LLM annotations. Under this configuration, identifier replacement performs 1.6× better, and the two mechanisms remain partially complementary.

5.9 Phase 1 Screening Funnel

Phase 1 excludes 66–93% of columns, and verification leaves only 3.5–7.6% renamed. Accepted columns gain 19.95–26.62 pp on their local queries. On BEAVER, Qwen3.5-27B commits zero; MiniMax-M2.7 commits four, but downstream impact remains bounded by the retrieval/assembly regime (§6.5). A direct 668-column probe of an additional "is improvement possible?" question would prune only 27 columns, including 4 of 80 execution-verified commits. It would discard 2.3% of verified gain while saving only 1.2% of Phase 3 work. Phase 1 therefore remains recall-oriented.

6 ANALYSIS AND INSIGHTS

We analyze where gains arise, why some systems benefit less, and the revision questions of PK–FK propagation, semantic grounding, query-blind proposals, cross-column composition, applicability, and refiner strength.

6.1 Database-Level: Coverage Predicts Improvement

Across 606 database-level configurations, every one of the 273 cases with no accepted rename has $\Delta = 0$. For coverage bins 0%, (0,3%], (3,7%], (7,15%], and >15%, query-weighted gains are respectively 0, +1.34, +0.60, +2.35, and +6.30 pp, with positive-cell rates 0%, 47%, 37%, 55%, and 62.5%. Overall, each coverage point corresponds to roughly +0.25–+0.40 pp ExAcc. BIRD's 3.51% column coverage thus explains its modest +0.19–+1.24 pp aggregate gains: few columns pass the execution threshold, rather than many accepted renames being ineffective.

6.2 Column-Level: LLM Top-1 is Unreliable

Phase 3 rejects the LLM's top choice in 60–80% of verified columns. When it disagrees, its selection beats top-1 roughly 8:1 on Dr.Spider-Abbr-27B and 35:1 on BIRD-27B; the remaining configurations range from 7.7:1 to 15.7:1. Most disagreements retain the original (62.7% of BIRD-27B verifications and all 106 BEAVER decisions), showing why an unchecked naturalness judgment over-renames schemas.

6.3 Query-Level: Flips Reveal Systematic Repair

For each query, C→W denotes a regression and W→C a repair. EGRefine has W→C≥C→W in 16/18 configurations (Table 15); on Dr.Spider-Abbr-27B, the ratio is 4.04 for C3 and 6.46 for DIN-SQL ($p < 0.001$), versus 1.37 for MAC-SQL ($p = 0.11$). Both ratios below one are MAC-SQL cells with margins of at most 13 queries.

LLM-Direct falls to 0.69/0.68 on BIRD-9B DIN/MAC. In all 12 matched cells, EGRefine breaks fewer queries (median reduction 53.5 per cell), so Phase 3's dominant effect is regression prevention.

6.4 Error Modes: Why MAC-SQL Benefits Least

MAC-SQL gains only +0.63 pp on Dr.Spider-Abbr versus +2.56/+2.48 for C3/DIN. This is neither name usage (86.1% versus 91.3/92.4%) nor dilution: on the 633 touching queries it still gains only +2.84 pp versus +11.53/+11.22. Its 7.7% regression rate is 2–4 times higher, and 73% of 49 regressions structurally rewrite the query (changed projection, dropped joins, aggregates, or predicates), whereas 77–79% of C3/DIN regressions retain query shape. Clearer names can therefore trigger an agent-level decomposition instability orthogonal to schema linking.

Table 15: Query-level flips per configuration (C→W = NoRef correct → refined wrong, a regression; W→C = repair; ratio = W→C / C→W). EGREFINE keeps the ratio above 1 in 11 of the 12 cells (16/18 including the BIRD+evidence variants); both exceptions are MAC-SQL cells with margins ≤13 queries. LLM-Direct drops below 1 in five cells.

			EGREFINE			LLM-Direct		
Bench	**Backbone**	**Algo**	C→W	W→C	ratio	C→W	W→C	ratio
ABBR	27B	C3	24	97	4.04	102	144	1.41
ABBR	27B	DIN	13	84	**6.46**	137	112	0.82
ABBR	27B	MAC	49	67	1.37	169	100	0.59
ABBR	9B	C3	67	180	2.69	160	177	1.11
ABBR	9B	DIN	47	195	4.15	145	192	1.32
ABBR	9B	MAC	85	145	1.71	166	184	1.11
BIRD	27B	C3	60	63	1.05	89	96	1.08
BIRD	27B	DIN	35	45	1.29	54	55	1.02
BIRD	27B	MAC	44	63	1.43	52	80	1.54
BIRD	9B	C3	74	97	1.31	81	73	0.90
BIRD	9B	DIN	52	75	1.44	81	56	0.69
BIRD	9B	MAC	63	56	0.89	69	47	0.68

Table 16: Query-subset decomposition. n_{tch} counts queries touching an accepted column; Δ_{tch} and Δ_{all} are ExAcc gains on touching and all queries.

Cell	**Algo**	n_{tch}	Δ_{tch}	Δ_{all}	**Ratio**
ABBR–27B	C3	633	+11.53	+2.56	4.50×
	DIN	633	+11.22	+2.48	4.52×
	MAC	633	+2.84	+0.63	4.51×
ABBR–9B	C3	1126	+10.04	+3.96	2.53×
	DIN	1126	+13.14	+5.19	2.53×
	MAC	1126	+5.33	+2.10	2.54×
BIRD–27B	C3	126	−1.59	+0.19	–
	DIN	126	+6.35	+0.65	9.77×
	MAC	126	−3.17	+1.24	–
BIRD–9B	C3	320	+7.19	+1.64	4.39×
	DIN	320	+5.62	+1.64	3.43×
27B→9B	DIN	633	+21.48	+8.95	2.40×

6.5 Benchmark-Level Scope: Why BEAVER Probes a Different Task

BEAVER's median gold SQL is 3,312 characters with four joins. BIRD and Dr.Spider average roughly 115–150 characters and 0–2 joins. About 70% of BEAVER queries have at least five joins and exceed 3,500 characters, while every solved query is below 400 characters with at most one join. The resulting 8–10% ceiling is retrieval/assembly-bound, leaving little name-level headroom (§6.11).

6.6 Query-Subset Decomposition: Concentration and Dilution

Restricting evaluation to queries that reference an accepted column exposes the local signal. Dr.Spider-27B gains concentrate by 4.5× for every algorithm (11.53/11.22/2.84 pp for C3/DIN/MAC); the 27B→9B DIN transfer reaches 21.48 pp. The touching fraction is only about 22% on Dr.Spider and 8% on BIRD, so coverage dilutes strong local effects. BIRD is also algorithm-dependent: DIN gains 6.35 pp on 126 touching queries, while C3/MAC do not, consistent with pruning and decomposition sensitivity to token changes.

6.7 Rule-Level: When Should a PK Rename Propagate?

Role-named FKs are common: 50.0% (179/358) of Dr.Spider and 59.4% (63/106) of BIRD FK edges differ from their referent. Unconditional propagation rewrites six such FKs in five main-run databases, sometimes replacing clear roles (e.g., Conductor_ID) with vague inherited names.

On those five databases (222 queries), preserving role names scores 66.7 vs. 65.3 on C3 and 57.7 vs. 57.7 on DIN, with no significant difference ($p = 0.61/1.00$). Uniformity therefore adds no detected join benefit and can erase context. The revised policy preserves these roles and limits propagation to same-name PK–FK ambiguity (§4.4.1).

6.8 Name-Level: Are the Selected Renames Semantically Grounded?

Dr.Spider-Abbr's pre-perturbation identifiers provide unseen human-written targets. Among 77 semantic commits, 35.1% exactly recover the target; mean token Jaccard rises 0.10→0.58, embedding cosine 0.31→0.79, and 89.6% move closer. Against user questions, 90% of Dr.Spider and 79% of BIRD commits increase embedding alignment (mean +0.23/+0.12). These canonical-target metrics directly measure semantic alignment; execution results determine whether that alignment improves the downstream task.

Alignment alone is insufficient: 91% of 269 rejected Dr.Spider columns also have a candidate with improved alignment (mean +0.27). For `ship.st`, plausible `ship_type` reduces ExAcc 0.5→0; for `battle.bg_cdr`, execution prefers `bulgar_commander` (0.83) over the more question-aligned `bulgarian_commander` (0.67). Thus accepted names are semantically grounded, but execution remains the operational discriminator.

Table 17: Candidate outcomes. Name-insensitive: all candidates tie the original; lexical proxy: headroom exists but no proposal shares question content words.

	ABBR	BIRD	BEAVER
Execution-verified columns	350	111	45
committed (best > original)	81 (23.1%)	28 (25.2%)	0
original retained	269 (76.9%)	83 (74.8%)	45 (100%)
Of original-retained cases:			
original already perfect	99 (36.8%)	10 (12.0%)	0
name-insensitive	149 (55.4%)	53 (63.9%)	43 (95.6%)
Lexical proxy (of verified)	28 (8.0%)	12 (10.8%)	—

Table 18: Forced-target query conditioning with shared Phase 3 scores. Held-out WC−standard ExAcc is all/touching queries; ABBR uses 70/30 and BIRD 50/50.

Benchmark	T	Targets	Novel	R^+	R^-	C3 held-out	DIN held-out
ABBR	0.7	28	40	4	1	0.00/0.00	0.00/0.00
ABBR	0	28	33	3	0	−0.34/−2.63	0.00/0.00
BIRD	0.7	10	15	0	0	0.00/0.00	0.00/0.00
BIRD	0	10	14	0	0	0.00/0.00	0.00/0.00

6.9 Generation-Level: What Does Query-Blind Candidate Generation Cost?

Phase 2 does not observe user queries; Phase 3 does. As Table 17 shows, a name that looks clear in isolation can still conflict with the vocabulary used in questions. Execution retains the original on 76.9%/74.8% of verified ABBR/BIRD columns, mainly because the original is already correct or all names score identically. Only 8.0%/10.8% have both execution headroom and no candidate sharing a question content word. This is an operational proxy for missed user vocabulary, not proof that conditioning can recover it.

Lexical overlap is also not a safe selector. Among original-retained cases with execution headroom, 78% on ABBR and 63% on BIRD already contain a candidate that shares user vocabulary but still fails verification. The `ship.st` case in §6.8 illustrates the risk: the apparently aligned `ship_type` is harmful. Query signals may therefore enlarge a candidate pool, but cannot replace execution-grounded selection. Conversely, for `cars_data.Edispl`, users themselves say "average edispl," so expanding the abbreviation can misalign with their vocabulary; the conservative rule keeps the original.

We pre-register the 28 ABBR and 12 BIRD proxy targets and force every target past Phase 1, removing screening as a confound. Standard and query-conditioned generation each propose three names; the conditioned arm sees the first three workload questions in $Q(c_i)$. We score the union of unique candidates once and select each arm from the same Phase 3 score dictionary. Thus target sets, original-name scores, and overlapping candidate scores are identical across arms. We test both the default temperature 0.7 and a deterministic temperature-0 sensitivity run.

Conditioning rescues 4/28 ABBR targets at $T = 0.7$ and 3/28 at $T = 0$, but 0/10 BIRD targets. On held-out execution, $T = 0.7$ ties both systems; $T = 0$ changes one C3 outcome and ties DIN (all $p = 1$). Thus query signals diversify the candidate pool but provide no held-out gain in this test. We retain the simpler query-blind generator and use execution-grounded selection to align its proposals with the workload objective.

Table 19: Five primary composition flips. Composed = isolation gain − spillover to the original + selected-name movement.

Column (database)	$\|Q\|$	Main	Spillover	Sel. move	Composed
`Treatments.cot` (dog_kennels_1)	12	+0.125	+0.208	0	−0.083
`Treatments.dot` (dog_kennels_4)	6	+0.167	+0.250	−0.083	−0.167
`Cartoon.wr` (tvshow_4)	6	+0.167	+0.167	−0.083	−0.083
`TV_Channel.cntry` (tvshow_5)	12	+0.125	+0.208	0	−0.083
`circuits.lng` (formula_1)	8	+0.063	+0.125	−0.063	−0.125

6.10 Composition-Level: Do Per-Column Gains Survive Joint Deployment?

Phase 3 scores each rename with all other names original, whereas deployment composes all accepted names. The main experiments already evaluate this composed schema end-to-end; the audit asks the stricter per-column question of whether the gain that justified each commit survives composition. This differs from sequential proposal feedback because Phase 2 generates every column's candidates from the same unmodified schema. The default Phase 2 generator is stochastic ($T = 0.7$); setting $T = 0$ would not remove this risk, because it arises from isolation scoring versus joint deployment.

For every commit, we rescore original and selected names both alone and with all other commits active, using an all-but-c_i baseline to separate spillover. The isolation gain is the difference observed with c_i renamed alone; the composed gain measures the same contrast after all other commits are active. All comparisons use one audit endpoint, distinct from the canonical serving checkpoint, so absolute scores are never mixed across the two.

The audit covers 109 commits; 16 single-rename cases are trivial. Among the 93 non-trivial columns, 49 replicate their isolation gain and form the primary set. Of these, 30 remain positive, 14 become neutral, and five flip negative. The remaining 44 differ at the audit endpoint and yield 13/24/7 positive/neutral/negative outcomes, so we report them separately. Table 19 shows that each primary flip combines +0.125−+0.250 spillover to the original score with at most one downward selected-name outcome. Each deficit corresponds to only one or two model-query outcomes. Composition is therefore susceptible, but it is not systematically harmful in this audit.

Reverting the five flips causes no detectable paired end-to-end change: Dr.Spider-Abbr moves −1.45 pp on C3 and +1.94 on DIN ($p = 0.55$ and 0.22); BIRD moves +2.88/−1.72 ($p = 0.33/0.58$). A from-scratch endpoint-and-sampling run stays within 0.6 pp on both algorithms. Strict deployments can add the composed audit at $2|\mathcal{M}||Q(c_i)|$ generations per column and revert flips.

6.11 Applicability and Scope: When Does Renaming Help?

Applicability depends on the source of errors, not on schema size alone (Table 20). A rename can help only when it improves schema selection or interpretation and downstream SQL generation remains

Table 20: Screening and refinement funnel. m: columns; n: Phase-1 output; n_r: accepted renames; Δ_{ref}: local gain on refined columns.

Cell	DB	m	n	n_r	Excl.	n_r/m	Δ_{ref}
ABBR–27B	90	1985	665	81	66.5%	4.08%	20.68
ABBR–9B	90	1985	841	151	57.6%	7.61%	20.94
BIRD–27B	11	798	251	28	68.5%	3.51%	26.62
BIRD+Ev.–27B	11	798	252	28	68.4%	3.51%	21.76
BIRD–9B	11	798	308	39	61.4%	4.89%	19.95
BEAVER–27B	4	1439	106	0	92.6%	0.00%	—
BEAVER–M2.7	4	1439	172	4	88.0%	0.28%	—

viable. This is a diagnostic, not a strict causal partition: names can also affect retrieval.

Renaming is most effective when names impede schema selection or interpretation, the Text-to-SQL stack is otherwise viable, and a representative workload supplies column-level support. These conditions hold on Dr.Spider-Abbr (+2.56/+2.48/+0.63 pp), Dr.Spider-Syn (+2.25 C3), and smaller verified gains on BIRD. The weakest tested workload (715 ABBR queries) also improves, but is not a universal threshold.

It is least effective when names are already clear, retrieval or assembly fails, or generation logic is wrong. In BEAVER, 96.3% of failures never assemble the gold column set, 2.9% reach it and fail in logic, and 0.8% truncate; 96% of verified columns are name-insensitive. The default refiner therefore commits zero names. A stronger refiner finds four and raises MAC-SQL 6.67→11.11% on covered databases, showing limited rather than impossible headroom.

Phase 3's name-insensitivity ratios are 55%, 64%, and 96% on ABBR, BIRD, and BEAVER. This workload-, candidate-, and verifier-dependent diagnostic is not a cutoff. Near-total insensitivity suggests improving retrieval or generation first. Conservative abstention is local to the refinement workload; unseen and composed effects remain empirical (§5.7, §6.10).

This diagnostic requires a viable Text-to-SQL stack. At near-zero baseline accuracy, identical candidate scores may reflect insufficient verifier resolution. It should be used with a query-level failure analysis and a small sensitivity probe, rather than as an automatic deployment gate. In practice, teams can run Phase 3 on a workload sample and inspect how often every candidate ties the original. If both name sensitivity and local execution headroom are absent, retrieval or SQL generation should be improved first.

The boundary shifts as retrieval and schema linking improve. Better retrieval moves errors from schema assembly toward interpretation, where renaming can help. A practical pipeline first retrieves the prompt-visible subset of a large schema and then applies EGRefine within that subset. This order also avoids refining columns that never enter the serving context.

6.12 Refiner-Side Model Strength: Does Frontier Reasoning Replace Execution?

Table 21 controls candidate generation: each frontier model's LLM-Direct and verified arms share screening and candidates. Without execution, Flash is indistinguishable from NoRef and Pro harms DIN by −3.86 pp ($p < 0.001$). Verification turns the same pools into +2.4–+2.9 pp gains ($p < 0.001$). Verified outcomes across Flash, Pro, and Qwen3.5-27B vary within about 1 pp despite different commit sets; execution, not refiner strength, is the stabilizing component.

Table 21: Refiner strength on Dr.Spider-Abbr. Each model's Direct/verified arms share screening and candidates; Δ is vs. same-endpoint NoRef and *** denotes McNemar $p < 0.001$.

Arm	C3	ΔC3	DIN-SQL	ΔDIN
NoRef	70.10	—	76.45	—
LLM-Direct (DeepSeek-V4-Flash)	70.38	+0.28 (n.s.)	76.94	+0.49 (n.s.)
LLM-Direct (DeepSeek-V4-Pro)	70.00	−0.11 (n.s.)	72.59	−3.86 ***
EGRefine (Flash candidates)	72.73	+2.63 ***	78.86	+2.41 ***
EGRefine (Pro candidates)	72.80	+2.70 ***	**79.39**	+2.94 ***
EGRefine (Qwen3.5-27B, anchor)	72.03	+1.93 ***	78.90	+2.45 ***

7 CONCLUSION

EGRefine formalizes Text-to-SQL schema refinement as a constrained optimization solved through execution-grounded verification, yielding a durable, drop-in schema. Across Dr.Spider, BIRD, and BEAVER, execution feedback is essential: it filters unreliable LLM preferences and produces refinements that transfer across models. The applicability and composition audits provide deployment criteria beyond aggregate accuracy.

Deployment. A cost-sensitive run can use $k = 2$, reducing Phase 3 calls by 25.1% from $k = 3$ without a detectable accuracy difference (Table 4). The verified VIEW can be reused across serving models (Tables 7 and 8) and rolled back without modifying base tables. Periodic drift checks can trigger offline re-refinement while the current mapping remains available.

Limitations. The refined schema is specific to its (schema, Q, $\mathcal{M}$) triple: gains depend on whether naming is the active bottleneck and vary by architecture (§6.4). Its execution-grounded verification requires a representative labeled workload—natural-language questions paired with gold SQL—to score candidates, the standard assumption of workload-driven physical design [6, 45]. Relaxing this to weak supervision (executed query logs without gold annotations, or NL-only self-consistency) is future work. The conservative guarantee is column- and workload-local; unseen queries and joint composition remain empirical, motivating the audits in §5.7 and §6.10. These diagnostics also identify when refinement should be deferred.

REFERENCES

[1] Serge Abiteboul, Richard Hull, and Victor Vianu. 1995. *Foundations of Databases*. Addison-Wesley.

[2] Sanjay Agrawal, Surajit Chaudhuri, and Vivek Narasayya. 2000. Automated Selection of Materialized Views and Indexes for SQL Databases. In *Proceedings of the 26th International Conference on Very Large Data Bases (VLDB)*.

[3] Adithya Bhaskar, Tushar Tomar, Ashutosh Sathe, and Sunita Sarawagi. 2023. Benchmarking and Improving Text-to-SQL Generation under Ambiguity. In *EMNLP*. 7053–7074.

[4] Shuaichen Chang, Jun Wang, Mingwen Dong, et al. 2023. Dr.Spider: A Diagnostic Evaluation Benchmark Towards Text-to-SQL Robustness. *arXiv preprint arXiv:2301.08881* (2023).

[5] Saumya Chaturvedi, Aman Chadha, and Laurent Bindschaedler. 2025. SQL-of-Thought: Multi-Agentic Text-to-SQL with Guided Error Correction. *arXiv preprint arXiv:2509.00581* (2025).

[6] Surajit Chaudhuri and Vivek Narasayya. 1997. An Efficient, Cost-Driven Index Selection Tool for Microsoft SQL Server. In *Proceedings of the 23rd International Conference on Very Large Data Bases (VLDB)*.

[7] Surajit Chaudhuri and Vivek Narasayya. 2007. Self-Tuning Database Systems: A Decade of Progress. In *Proceedings of the International Conference on Very Large Data Bases (VLDB)*. 3–14.

[8] Bei Chen, Fengji Zhang, Anh Nguyen, et al. 2023. CodeT: Code Generation with Generated Tests. *arXiv preprint arXiv:2207.10397* (2023).

[9] Kaiwen Chen, Yueting Chen, Xiaohui Yu, and Nick Koudas. 2025. Reliable Text-to-SQL with Adaptive Abstention. *arXiv preprint arXiv:2501.10858* (2025).

[10] Peter Baile Chen, Michael Cafarella, Çağatay Demiralp, and Michael Stonebraker. 2024. BEAVER: An Enterprise Benchmark for Text-to-SQL. *arXiv preprint arXiv:2409.02038* (2024).

[11] Xinyun Chen, Maxwell Lin, Nathanael Schärli, and Denny Zhou. 2024. Teaching Large Language Models to Self-Debug. *arXiv preprint arXiv:2304.05128* (2024).

[12] Ziru Chen, Shijie Chen, Michael White, Raymond Mooney, et al. 2023. Text-to-SQL Error Correction with Language Models of Code. In *ACL (Short Papers)*. 1359–1372.

[13] Xuemei Dong, Chao Zhang, Yuxiang Ge, Yun Mao, Yongkang Gao, Jian Lin, and Dongfang Lou. 2023. C3: Zero-shot Text-to-SQL with ChatGPT. *arXiv preprint arXiv:2307.07306* (2023).

[14] Ahmed Elgohary, Saghar Hosseini, and Ahmed Hassan Awadallah. 2020. Speak to Your Parser: Interactive Text-to-SQL with Natural Language Feedback. In *ACL*. 2065–2077.

[15] Jonathan Fürst, Catherine Kosten, Farhad Nooralahzadeh, et al. 2024. Evaluating the Data Model Robustness of Text-to-SQL Systems Based on Real User Queries. *arXiv preprint arXiv:2402.08349* (2024).

[16] Yujian Gan, Xinyun Chen, Qiuping Huang, Matthew Purver, John R. Woodward, Jinxia Xie, and Pengsheng Huang. 2021. Towards Robustness of Text-to-SQL Models against Synonym Substitution. In *ACL*. 2505–2515.

[17] Yujian Gan, Xinyun Chen, and Matthew Purver. 2021. Exploring Underexplored Limitations of Cross-Domain Text-to-SQL Generalization. In *EMNLP*. 8926–8931.

[18] Dawei Gao, Haibin Wang, Yaliang Li, et al. 2023. Text-to-SQL Empowered by Large Language Models: A Benchmark Evaluation. *arXiv preprint arXiv:2308.15363* (2023).

[19] Hector Garcia-Molina, Jeffrey D. Ullman, and Jennifer Widom. 2008. *Database Systems: The Complete Book* (2nd ed.). Pearson Prentice Hall.

[20] Zijin Hong, Zheng Yuan, Qinggang Zhang, et al. 2025. Next-Generation Database Interfaces: A Survey of LLM-Based Text-to-SQL. *arXiv preprint arXiv:2406.08426* (2025).

[21] Klaus Jansen and Petra Scheffler. 1997. Generalized coloring for tree-like graphs. *Discrete Applied Mathematics* 75, 2 (1997), 135–155.

[22] George Katsogiannis-Meimarakis and Georgia Koutrika. 2023. A Survey on Deep Learning Approaches for Text-to-SQL. *The VLDB Journal* 32, 4 (2023), 905–936.

[23] Fangyu Lei et al. 2025. Spider 2.0: Evaluating Language Models on Real-World Enterprise Text-to-SQL Workflows. In *ICLR*.

[24] Haoyang Li, Binyuan Hui, Ge Qu, Jiaxi Yang, Binhua Li, Bowen Li, Bailin Wang, Bowen Qin, Rongyu Cao, and Junliang Li. 2023. RESDSQL: Decoupling Schema Linking and Skeleton Parsing for Text-to-SQL. In *Proc. of AAAI*.

[25] Haoyang Li, Jing Zhang, Hanbing Liu, et al. 2024. CodeS: Towards Building Open-Source Language Models for Text-to-SQL. *Proceedings of the ACM on Management of Data* 2, 3 (2024), 1–28.

[26] Jinyang Li, Binyuan Hui, Ge Qu, et al. 2023. Can LLM Already Serve as a Database Interface? A Big Bench for Large-Scale Database Grounded Text-to-SQLs. In *Advances in Neural Information Processing Systems (NeurIPS)*, Vol. 36.

[27] Xi Victoria Lin, Richard Socher, and Caiming Xiong. 2020. Bridging Textual and Tabular Data for Cross-Domain Text-to-SQL Semantic Parsing. In *Findings of EMNLP*. 4870–4888.

[28] Yuyu Luo, Guoliang Li, Ju Fan, Chengliang Chai, and Nan Tang. 2025. Natural Language to SQL: State of the Art and Open Problems. *Proceedings of the VLDB Endowment* 18, 12 (2025), 5466–5471.

[29] Kyle Luoma and Arun Kumar. 2025. SNAILS: Schema Naming Assessments for Improved LLM-Based SQL Inference. *Proceedings of the ACM on Management of Data* 3, 1 (2025), 1–25.

[30] Karime Maamari, Fadhil Abubaker, Daniel Jaroslawicz, and Amine Mhedhbi. 2024. The Death of Schema Linking? Text-to-SQL in the Age of Well-Reasoned Language Models. *arXiv preprint arXiv:2408.07702* (2024).

[31] Wuwei Mao et al. 2024. Enhancing Text-to-SQL Parsing through Question Rewriting and Execution-Guided Refinement. In *Findings of ACL*. 2009–2024.

[32] Andrew Pavlo, Gustavo Angulo, Joy Arulraj, Haibin Lin, Jiexi Lin, Lin Ma, Prashanth Menon, Todd C. Mowry, Matthew Perron, Ian Quah, Siddharth Santurkar, Anthony Tomasic, Skye Toor, Dana Van Aken, Ziqi Wang, Yingjun Wu, Ran Xian, and Tieying Zhang. 2017. Self-Driving Database Management Systems. In *8th Biennial Conference on Innovative Data Systems Research (CIDR)*.

[33] Mohammadreza Pourreza and Davood Rafiei. 2023. DIN-SQL: Decomposed In-Context Learning of Text-to-SQL with Self-Correction. In *NeurIPS*. 36339–36348.

[34] Luyu Qiu, Jianing Li, Chi Su, and Lei Chen. 2025. Interactive Text-to-SQL via Expected Information Gain for Disambiguation. *arXiv preprint arXiv:2507.06467* (2025).

[35] Sahil Qiu et al. 2025. PRACTIQ: A Practical Conversational Text-to-SQL Dataset with Ambiguous and Unanswerable Queries. In *NAACL*.

[36] Ge Qu, Jinyang Li, Bowen Qin, et al. 2025. SHARE: An SLM-Based Hierarchical Action Correction Assistant for Text-to-SQL. In *ACL*. 11268–11292.

[37] Cedric Renggli, Ihab F. Ilyas, and Theodoros Rekatsinas. 2025. Fundamental Challenges in Evaluating Text2SQL Solutions and Detecting Their Limitations. *arXiv preprint arXiv:2501.18197* (2025).

[38] Irina Saparina and Mirella Lapata. 2024. AMBROSIA: A Benchmark for Parsing Ambiguous Questions into Database Queries. In *NeurIPS*. 90600–90628.

[39] Torsten Scholak, Nathan Schucher, and Dzmitry Bahdanau. 2021. PICARD: Parsing Incrementally for Constrained Auto-Regressive Decoding from Language Models. In *EMNLP*. 9895–9901.

[40] Noah Shinn, Federico Cassano, Ashwin Gopinath, Karthik Narasimhan, and Shunyu Yao. 2023. Reflexion: Language Agents with Verbal Reinforcement Learning. In *NeurIPS*.

[41] Shayan Talaei, Mohammadreza Pourreza, Yu-Chen Chang, Azalia Mirhoseini, and Amin Saberi. 2024. CHESS: Contextual Harnessing for Efficient SQL Synthesis. *arXiv preprint arXiv:2405.16755* (2024).

[42] Yuan Tian, Zheng Zhang, Zheng Ning, et al. 2023. Interactive Text-to-SQL Generation via Editable Step-by-Step Explanations. In *EMNLP*. 16149–16166.

[43] Kapil Vaidya, Abishek Sankararaman, Jialin Ding, Chuan Lei, Xiao Qin, Balakrishnan Narayanaswamy, and Tim Kraska. 2025. ODIN: A NL2SQL Recommender to Handle Schema Ambiguity. *arXiv preprint arXiv:2505.19302* (2025).

[44] Gary Valentin, Michael Zuliani, Daniel C. Zilio, Guy M. Lohman, and Alan Skelley. 2000. DB2 Advisor: An Optimizer Smart Enough to Recommend Its Own Indexes. In *Proceedings of the International Conference on Data Engineering (ICDE)*. 101–110.

[45] Dana Van Aken, Andrew Pavlo, Geoffrey J. Gordon, and Bohan Zhang. 2017. Automatic Database Management System Tuning Through Large-Scale Machine Learning. In *Proceedings of the 2017 ACM International Conference on Management of Data (SIGMOD)*. 1009–1024.

[46] Bing Wang, Yan Gao, Zhoujun Li, and Jian-Guang Lou. 2023. Know What I Don't Know: Handling Ambiguous and Unknown Questions for Text-to-SQL. In *Findings of ACL*. 5701–5714.

[47] Bing Wang, Changyu Ren, Jian Yang, et al. 2025. MAC-SQL: A Multi-Agent Collaborative Framework for Text-to-SQL. In *COLING*. 540–557.

[48] Bailin Wang, Richard Shin, Xiaodong Liu, Oleksandr Polozov, and Matthew Richardson. 2020. RAT-SQL: Relation-Aware Schema Encoding and Linking for Text-to-SQL Parsers. In *ACL*. 7567–7578.

[49] Fabian Wenz, Omar Bouattour, Devin Yang, Justin Choi, Cecil Gregg, Nesime Tatbul, and Çağatay Demiralp. 2026. BenchPress: A Human-in-the-Loop Annotation System for Rapid Text-to-SQL Benchmark Curation. In *CIDR*.

[50] Liu Xinyu, Shen Shuyu, Li Boyan, et al. 2025. A Survey of Text-to-SQL in the Era of LLMs: Where Are We, and Where Are We Going? *arXiv preprint arXiv:2408.05109* (2025).

[51] Tao Yu, Rui Zhang, Heyang Er, et al. 2019. CoSQL: A Conversational Text-to-SQL Challenge Towards Cross-Domain Natural Language Interfaces to Databases. In *EMNLP-IJCNLP*. 1962–1979.

[52] Tao Yu, Rui Zhang, Kai Yang, Michihiro Yasunaga, et al. 2018. Spider: A Large-Scale Human-Labeled Dataset for Complex and Cross-Domain Semantic Parsing and Text-to-SQL Task. In *EMNLP*. 3911–3921.

[53] Meng Zhang, Kexin Ma, Liyang Xu, Kedi Zhang, Yuanxi Peng, and Ruochun Jin. 2025. CLEAR: A Parser-Independent Disambiguation Framework for NL2SQL. In *ICDE*. 1–14.

[54] Victor Zhong, Caiming Xiong, and Richard Socher. 2017. Seq2SQL: Generating Structured Queries from Natural Language Using Reinforcement Learning. *arXiv preprint arXiv:1709.00103* (2017).